Frontend Technical Documentation

Next.js customer/chef web architecture and flows.

This document explains its subject first in plain language, then in engineering detail. It is grounded in the Craves repository snapshot and deliberately separates current implementation, legacy/reference code, milestone foundations and repository gaps so readers do not mistake aspiration for proof.

Version 1.0 | Evidence date: 14 August 2026 | Repository: rmorampudi09-arch/Craves-Build-platform | Snapshot commit: 3225f7fa531a6b07185fb5e034f7930f8bd8b571

Next.js App Router - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

Next.js App Router - technical architecture

Plain-English explanation

The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules.

How it works technically

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The frontend boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Snapshot: main @ 3225f7fa531a (14 August 2026).

Next.js App Router - end-to-end flow

Plain-English explanation

The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules.

How it works technically

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The frontend boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Snapshot: main @ 3225f7fa531a (14 August 2026).

Next.js App Router - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

Next.js App Router - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

same-origin BFF - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

same-origin BFF - technical architecture

Plain-English explanation

The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules.

How it works technically

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The frontend boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Snapshot: main @ 3225f7fa531a (14 August 2026).

same-origin BFF - end-to-end flow

Plain-English explanation

The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules.

How it works technically

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The frontend boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Snapshot: main @ 3225f7fa531a (14 August 2026).

same-origin BFF - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

same-origin BFF - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

phone OTP - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

phone OTP - technical architecture

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

phone OTP - end-to-end flow

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

phone OTP - errors and edge cases

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

phone OTP - deployment, security and operational validation

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

secure session cookies - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

secure session cookies - technical architecture

Plain-English explanation

The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules.

How it works technically

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The frontend boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Snapshot: main @ 3225f7fa531a (14 August 2026).

secure session cookies - end-to-end flow

Plain-English explanation

The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules.

How it works technically

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The frontend boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Snapshot: main @ 3225f7fa531a (14 August 2026).

secure session cookies - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

secure session cookies - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

home/discovery - plain-language purpose

Plain-English explanation

Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation.

How it works technically

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The catalog boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

home/discovery - technical architecture

Plain-English explanation

Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation.

How it works technically

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The catalog boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

home/discovery - end-to-end flow

Plain-English explanation

Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation.

How it works technically

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The catalog boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

home/discovery - errors and edge cases

Plain-English explanation

Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation.

How it works technically

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The catalog boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

home/discovery - deployment, security and operational validation

Plain-English explanation

Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation.

How it works technically

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The catalog boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

search and meal details - plain-language purpose

Plain-English explanation

Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation.

How it works technically

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The catalog boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

search and meal details - technical architecture

Plain-English explanation

Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation.

How it works technically

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The catalog boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

search and meal details - end-to-end flow

Plain-English explanation

Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation.

How it works technically

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The catalog boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

search and meal details - errors and edge cases

Plain-English explanation

Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation.

How it works technically

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The catalog boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

search and meal details - deployment, security and operational validation

Plain-English explanation

Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation.

How it works technically

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The catalog boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

cart and quantity - plain-language purpose

Plain-English explanation

Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment.

How it works technically

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The checkout boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

cart and quantity - technical architecture

Plain-English explanation

Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment.

How it works technically

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The checkout boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

cart and quantity - end-to-end flow

Plain-English explanation

Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment.

How it works technically

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The checkout boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

cart and quantity - errors and edge cases

Plain-English explanation

Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment.

How it works technically

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The checkout boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

cart and quantity - deployment, security and operational validation

Plain-English explanation

Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment.

How it works technically

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The checkout boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

checkout quote - plain-language purpose

Plain-English explanation

Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment.

How it works technically

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The checkout boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

checkout quote - technical architecture

Plain-English explanation

Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment.

How it works technically

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The checkout boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

checkout quote - end-to-end flow

Plain-English explanation

Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment.

How it works technically

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The checkout boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

checkout quote - errors and edge cases

Plain-English explanation

Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment.

How it works technically

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The checkout boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

checkout quote - deployment, security and operational validation

Plain-English explanation

Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment.

How it works technically

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The checkout boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

address/location - plain-language purpose

Plain-English explanation

Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails.

How it works technically

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The location boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

address/location - technical architecture

Plain-English explanation

Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails.

How it works technically

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The location boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

address/location - end-to-end flow

Plain-English explanation

Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails.

How it works technically

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The location boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

address/location - errors and edge cases

Plain-English explanation

Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails.

How it works technically

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The location boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

address/location - deployment, security and operational validation

Plain-English explanation

Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails.

How it works technically

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The location boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

Cashfree payment - plain-language purpose

Plain-English explanation

Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state.

How it works technically

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The payment boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

Cashfree payment - technical architecture

Plain-English explanation

Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state.

How it works technically

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The payment boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

Cashfree payment - end-to-end flow

Plain-English explanation

Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state.

How it works technically

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The payment boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

Cashfree payment - errors and edge cases

Plain-English explanation

Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state.

How it works technically

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The payment boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

Cashfree payment - deployment, security and operational validation

Plain-English explanation

Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state.

How it works technically

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The payment boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

confirmation/history/tracking - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

confirmation/history/tracking - technical architecture

Plain-English explanation

The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules.

How it works technically

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The frontend boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Snapshot: main @ 3225f7fa531a (14 August 2026).

confirmation/history/tracking - end-to-end flow

Plain-English explanation

The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules.

How it works technically

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The frontend boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Snapshot: main @ 3225f7fa531a (14 August 2026).

confirmation/history/tracking - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

confirmation/history/tracking - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

• Stale UI/cache must not override server state. • UI visibility is not authorization evidence. • Retries of money/state-changing operations require idempotency awareness. • Provider/configuration outages should preserve core domain consistency where safe. • Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).